

FIAP · NAVEGANDO PELO MUNDO DEVOPS

InovaGAB

Pipeline CI/CD, containerização e orquestração
da API de inovação corporativa

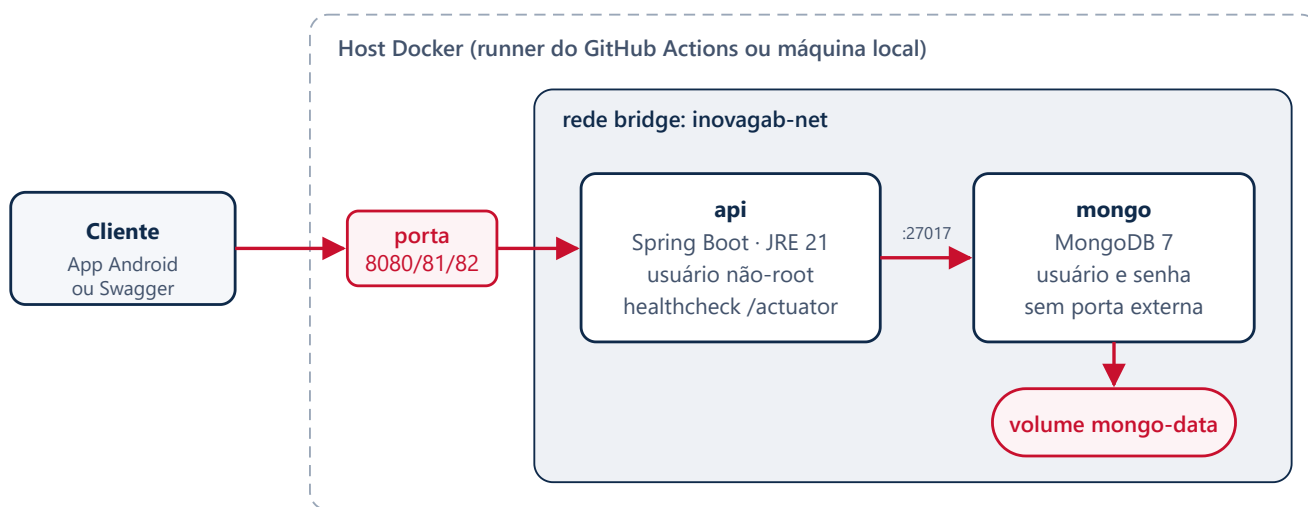
Integrante	Vitor Hugo — RM 559349
Projeto base	API REST InovaGAB (Java 21 · Spring Boot 3.3 · MongoDB 7)
Ferramenta CI/CD	GitHub Actions
Orquestração	Docker Compose
Ambientes	Staging e Produção

1. Visão geral e arquitetura

O InovaGAB é a plataforma de inovação corporativa do Grupo Água Branca: operadores enviam ideias, gestores fazem a curadoria e transformam ideias em projetos, e líderes definem orientações estratégicas e acompanham o portfólio em um dashboard. A API foi desenvolvida nas sprints anteriores em Java Spring Boot com MongoDB.

Neste desafio a aplicação foi preparada para um fluxo de entrega contínua: o código é compilado e testado automaticamente a cada alteração, empacotado em uma imagem Docker versionada e implantado em dois ambientes — **staging** e **produção** — com validação automática após cada deploy.

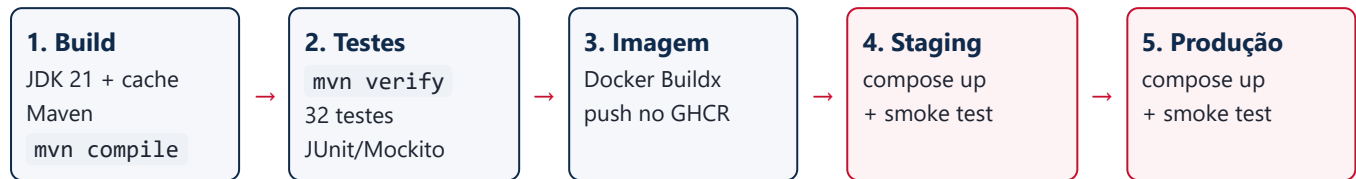
Arquitetura de execução



Componente	Responsabilidade
api	API REST (autenticação JWT, ideias, projetos, pontos, dashboard, auditoria, IA). Exposta na porta do ambiente.
mongo	Banco de dados. Acessível apenas pela rede interna <code>inovagab-net</code> , com usuário e senha.
mongo-data	Volume nomeado que persiste os dados do MongoDB entre reinícios dos containers.
GHCR	GitHub Container Registry: guarda as imagens versionadas geradas pelo pipeline.

2. Pipeline CI/CD

Ferramenta: GitHub Actions. O pipeline está em `.github/workflows/ci-cd.yml` e o deploy, comum aos dois ambientes, em `.github/workflows/deploy.yml` (workflow reutilizável). Usar o mesmo workflow para staging e produção garante que os dois ambientes são implantados pelos mesmos passos.



Etapas

Etapa	O que acontece
1. Build	Checkout, instalação do JDK 21 (Temurin) com cache do repositório Maven e compilação. Um erro de compilação interrompe o pipeline.
2. Testes automatizados	<code>mvn verify</code> executa os 32 testes das regras de negócio (pontuação, dupla aprovação, prioridade, ROI, agregados do dashboard). O resumo vai para a página da execução e os relatórios Surefire e o <code>.jar</code> ficam disponíveis como artefatos por 14 dias.
3. Imagem Docker	Build multi-stage com cache de camadas (<code>type=gha</code>) e publicação no GHCR com as tags <code>sha-<commit></code> , <code><branch></code> e <code>latest</code> . Em pull requests a imagem é apenas construída, validando o Dockerfile sem publicar.
4. Deploy staging	Baixa a imagem <code>sha-<commit></code> , sobe API + MongoDB com <code>env/staging.env</code> e os secrets do environment <code>staging</code> , aguarda os healthchecks (<code>--wait</code>) e roda o smoke test.
5. Deploy produção	Só na branch <code>main</code> e só se staging passou. Mesma imagem, com <code>env/production.env</code> e os secrets do environment <code>production</code> . Pode exigir aprovação manual (Required reviewers).

Lógica de disparo

Evento	Build	Testes	Imagem	Staging	Produção
Pull request para <code>main</code>	✓	✓	só build	—	—
Push em <code>develop</code>	✓	✓	✓	✓	—
Push em <code>main</code>	✓	✓	✓	✓	✓

Smoke test pós-deploy

`scripts/smoke-test.sh` roda contra o ambiente recém-implantado e falha o pipeline se qualquer verificação não passar:

- `/actuator/health` com status `UP` e o MongoDB conectado;
- `/actuator/info` informando o ambiente esperado (`staging` ou `production`) e a versão implantada;

- login com usuário de teste devolvendo token JWT;
- `/api/auth/me` e `/api/orientacoes` autenticados (leitura real no banco);
- requisição sem token bloqueada com HTTP 401.

Promoção da mesma imagem: staging e produção usam exatamente a imagem `sha-<commit>` construída na etapa 3. O artefato validado em staging é o mesmo que chega à produção — não há rebuild entre ambientes.

Onde os ambientes rodam: cada deploy sobe o ambiente em um runner Linux dedicado do GitHub Actions, valida e encerra ao final. Staging e produção rodam em máquinas separadas, com configuração e segredos separados por GitHub Environment. Para um servidor permanente, basta trocar `runs-on` por um *self-hosted runner* instalado no servidor.

3. Docker: imagem, orquestração e comandos

Dockerfile (multi-stage)

```
# ----- Estagio 1: build -----
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /build
COPY pom.xml .
RUN mvn -B -q dependency:go-offline          # dependências em cache
COPY src ./src
RUN mvn -B -q package -DskipTests \
    && cp target/inovagab-backend-*.jar app.jar

# ----- Estagio 2: runtime -----
FROM eclipse-temurin:21-jre-alpine
RUN addgroup -S inovagab && adduser -S inovagab -G inovagab
WORKDIR /app
COPY --from=build --chown=inovagab:inovagab /build/app.jar app.jar
USER inovagab                                # não-root
ENV SERVER_ADDRESS=0.0.0.0 SERVER_PORT=8080 \
    JAVA_OPTS="-XX:MaxRAMPercentage=75.0 -XX:+UseContainerSupport"
EXPOSE 8080
HEALTHCHECK --interval=15s --timeout=5s --start-period=60s --retries=5 \
    CMD wget -qO- http://127.0.0.1:8080/actuator/health/liveness || exit 1
ENTRYPOINT ["sh", "-c", "exec java $JAVA_OPTS -jar /app/app.jar"]
```

Imagem criada

Item	Valor
Registro	ghcr.io/<usuario>/<repositorio>/inovagab-backend
Tags	sha-<commit> (imutável, usada no deploy), main / develop , latest
Base de execução	eclipse-temurin:21-jre-alpine — sem Maven nem JDK na imagem final
Usuário	inovagab (não-root)
Porta	8080

docker-compose.yml

Recurso	Uso
Serviços	api e mongo
Rede	inovagab-net (bridge). O MongoDB não publica porta: só a API o acessa pelo nome mongo .
Volume	mongo-data em /data/db
Variáveis de ambiente	SPRING_PROFILES_ACTIVE , APP_AMBIENTE , APP_VERSION , MONGODB_URI (com usuário/senha), JWT_SECRET , SEED_ENABLED , GEMINI_API_KEY , JAVA_OPTS
Ordem de subida	depends_on COM condition: service_healthy
Isolamento por ambiente	Cada ambiente sobe como um projeto Compose próprio (-p inovagab-staging / -p inovagab-production), com rede, volume e porta separados.

Configuração por ambiente

	Local	Staging	Produção
Arquivo	.env	env/staging.env	env/production.env
Perfil Spring	default	staging	prod
Porta	8080	8081	8082
Log	DEBUG	DEBUG	INFO
Validade do JWT	480 min	480 min	120 min
Segredos	valores de desenvolvimento	secrets do environment staging	secrets do environment production

Comandos utilizados

```
# local: build da imagem e subida de API + banco
docker compose up -d --build
docker compose ps
docker compose logs -f api
docker compose down          # mantém os dados | down -v apaga o volume

# ambientes isolados (os mesmos usados no pipeline)
docker compose -p inovagab-staging --env-file env/staging.env up -d --build
docker compose -p inovagab-production --env-file env/production.env up -d --build

# no pipeline (deploy.yml)
docker compose -p inovagab-staging --env-file env/staging.env pull
docker compose -p inovagab-staging --env-file env/staging.env up -d --no-build --wait
./scripts/smoke-test.sh http://localhost:8081 staging
```

4. Evidências: pipeline em execução

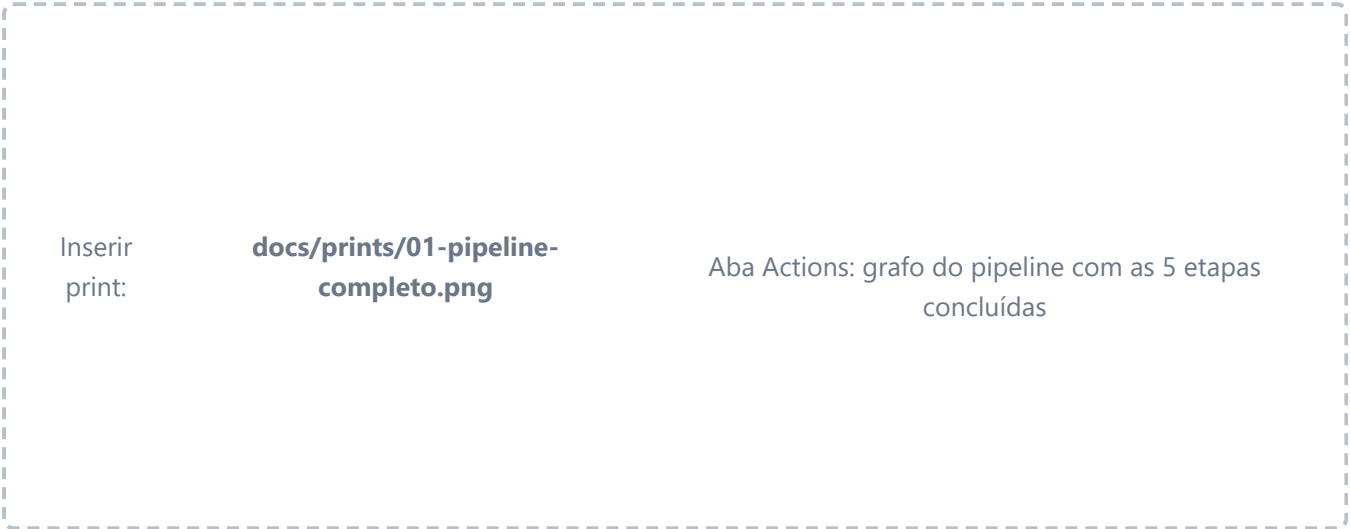


Figura 1 — Pipeline completo na branch `main`: build, testes, imagem, staging e produção.



Figura 2 — Etapa de build automática.



Figura 3 — Testes automatizados executados no pipeline.

Inserir
print:

**docs/prints/04-imagem-
docker.png**

Página Packages do GitHub com a imagem inovagab-
backend

Figura 4 — Imagem publicada no GitHub Container Registry.

5. Evidências: staging e produção funcionando

Inserir
print:

**docs/prints/05-deploy-
staging.png**

Summary do job Deploy em staging: containers, health, info e
smoke test

Figura 5 — Deploy em staging: containers saudáveis, `/actuator/info` com `ambiente: staging` e smoke test aprovado.

Inserir
print:

**docs/prints/06-deploy-
producao.png**

Summary do job Deploy em production: containers, health, info e
smoke test

Figura 6 — Deploy em produção: `ambiente: production` e smoke test aprovado.

Inserir
print:

**docs/prints/07-
environments.png**

Settings → Environments ou painel Deployments do
repositório

Figura 7 — Ambientes `staging` e `production` registrados no GitHub.

6. Desafios encontrados e soluções

Desafio	Como foi resolvido
A API escutava apenas em <code>127.0.0.1</code> , o que impede acesso de fora do container.	O endereço passou a vir da variável <code>SERVER_ADDRESS</code> : continua <code>127.0.0.1</code> na execução local sem Docker e o Dockerfile define <code>0.0.0.0</code> .
A chave da API do Gemini estava escrita no <code>application.yml</code> , ou seja, iria para o repositório e para a imagem.	A chave foi removida do código. Agora vem de <code>GEMINI_API_KEY</code> (arquivo <code>.env</code> local, fora do Git, ou secret do GitHub Environment). Sem ela, os endpoints de IA respondem 503 e o resto da API funciona.
O MongoDB rodava sem autenticação e com porta aberta no host.	O banco agora tem usuário e senha e não publica porta: só a API o acessa pela rede interna <code>inovagab-net</code> .
Sem controle de ordem, a API pode subir antes de o banco aceitar conexões e falhar na carga inicial de dados.	Healthcheck no MongoDB (<code>mongosh ping</code>) e <code>depends_on: condition: service_healthy</code> na API. No pipeline, <code>docker compose up --wait</code> só segue quando os dois estão saudáveis.
Garantir que produção recebe exatamente o que foi testado em staging.	A imagem é construída uma única vez com a tag imutável <code>sha-<commit></code> e promovida para os dois ambientes. Produção depende do sucesso de staging.
Ter configuração e segredos diferentes por ambiente sem duplicar o pipeline.	Um único <code>docker-compose.yml</code> parametrizado, arquivos <code>env/staging.env</code> e <code>env/production.env</code> , perfis Spring <code>staging</code> e <code>prod</code> , GitHub Environments para os secrets e um workflow de deploy reutilizável.
Comprovar que o deploy realmente funcionou, e não apenas que o container subiu.	Smoke test pós-deploy com login JWT, leitura autenticada no banco, verificação do ambiente via <code>/actuator/info</code> e checagem de segurança (401 sem token).
O registro GHCR exige nome de imagem em minúsculas, e o nome do repositório pode ter maiúsculas.	O nome é convertido com <code>\${GITHUB_REPOSITORY,,}</code> no passo que define as tags.
A máquina de desenvolvimento tinha Java 17, mas o projeto exige Java 21.	A compilação foi levada para dentro do container (estágio <code>maven:3.9-eclipse-temurin-21</code>) e para o runner do pipeline, eliminando a dependência da versão instalada localmente.

7. Checklist de entrega

Item	OK
Projeto compactado em .ZIP com estrutura organizada	☒
Dockerfile funcional	☒
docker-compose.yml ou arquivos Kubernetes	☒
Pipeline com etapas de build, teste e deploy	☒
README.md com instruções e prints	☒
Documentação técnica com evidências (PDF ou PPT)	☒
Deploy realizado nos ambientes staging e produção	☒

Tecnologias utilizadas

Categoria	Tecnologia
Linguagem / framework	Java 21, Spring Boot 3.3 (Web, Security, Data MongoDB, Validation, Actuator)
Banco de dados	MongoDB 7
Testes	JUnit 5, Mockito, Spring Boot Test
Build	Maven 3.9
Containerização / orquestração	Docker (multi-stage), Docker Compose v2
CI/CD	GitHub Actions, GitHub Environments, GitHub Container Registry